

SRRV Service Platform Turnover Documentation

Technical, deployment, database, API, access, maintenance, and support documentation for the SRRV Service Platform.

SYSTEM	Special Resident Retiree's Visa (SRRV) Service Platform
PREPARED FOR	Client turnover / handover
CLIENT / PROJECT OWNER	Nestor Licanto
CONTRACT REFERENCE	Project-Base Web Development Contract, effective May 22, 2026 – July 30, 2026
PRODUCTION DOMAIN	<code>retirewellsrv.com</code>
BASIS	Application behaviour as implemented on branch <code>develop</code>
COMPANION DOCUMENTS	User Guide, Admin & Super Admin Guide, and the Sign-off / Acceptance Certificate are separate documents — see the last page of this document for links.

Contents

2	Technical & System Documentation
3	Deployment Guide
4	Environment Variables
5	Database Schema / ERD
6	API Documentation
7	Access Handover Sheet
8	Credentials Transfer Checklist
10	Maintenance Guide
11	Source Code Repository Access
12	Known Issues / Limitations
13	Support / Warranty Terms

Numbering follows the client's original turnover checklist. Items 1 (User Guide), 9 (Admin & Super Admin Guide), and 14 (Sign-off / Acceptance Certificate) are each their own standalone document — see the closing page for links.

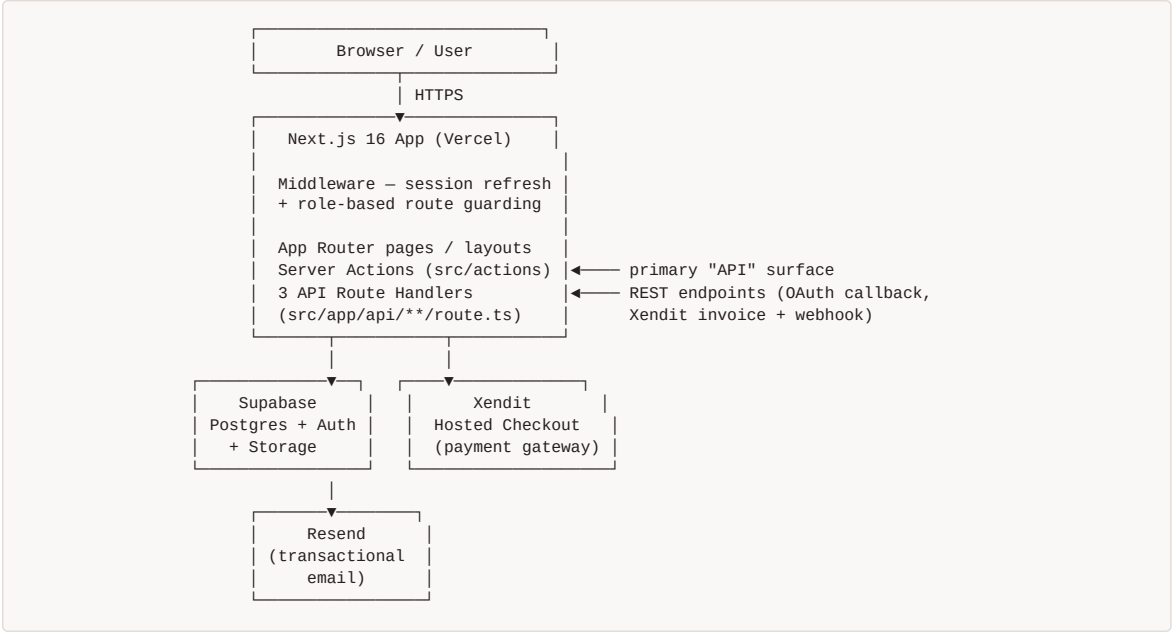
2. Technical & System Documentation

Architecture, technology stack, project structure, and the design decisions behind the SRRV Service Platform.

Architecture Overview

The SRRV Service Platform is a single Next.js application (no separate backend service). It serves three audiences from one codebase:

- **Public site** — informational pages, no auth required.
- **Applicant portal** (/applicant/*) — signed-in retirees managing their SRRV application.
- **Admin portal** (/admin/*) — staff reviewing consultations, applications, documents, payments.
- **Super Admin portal** (/super-admin/*) — manages admin accounts, read-only client register.



Server Actions (not REST endpoints) are the primary way the UI talks to the backend — this is a Next.js App Router convention, not a gap. See §6 (API Documentation) for the full inventory of both.

Tech Stack

Layer	Choice	Version
Framework	Next.js (App Router)	16.2.6
UI library	React	19.2.6
Language	TypeScript	5.x
Styling	Tailwind CSS + shadcn/ui (Radix primitives)	Tailwind 3.4, shadcn 4.8
Forms/validation	react-hook-form + Zod	RHF 7.76, Zod 4.4
Data fetching (client)	TanStack Query	5.100
Database & Auth	Supabase (Postgres, Auth, Storage) via @supabase/ssr	supabase-js 2.106
Payments	Xendit (hosted checkout) via xendit-node	7.0
Email	Resend (raw REST call, no SDK dependency)	—
Charts	Recharts	3.9
PDF viewing	pdfjs-dist / react-pdf	5.4 / 10.4
Testing	Vitest + Testing Library + jsdom	Vitest 2.x
Package manager	pnpm	—
Linting/formatting	ESLint (eslint-config-next) 10, Biome, Prettier, Husky + lint-staged	—
Hosting	Vercel	—

Full pinned versions: package.json at the repository root.

Folder / Project Structure

```
src/
  app/
    (auth)/          Public auth routes – login, register, forgot-password
    (public)/        Public marketing pages
    (protected)/
      admin/         Admin portal (role-gated layout)
      applicant/     Applicant portal (role-gated layout)
      super-admin/   Super admin portal (role-gated layout)
    api/
      auth/callback/route.ts    Supabase OAuth/email-confirmation callback
      payments/invoice/route.ts Creates a Xendit hosted-checkout invoice
      webhooks/xendit/route.ts  Receives Xendit payment status callbacks
  actions/          Server Actions – the application/business logic layer
  admin/            Admin-only actions (dashboard, applications, consultations,
                    documents, payments, services, admin/user management)
  applicant/        Applicant-only actions (application, consultation, document,
                    notifications, profile)
  auth.ts           Login/register/reset-password actions
  components/
    ui/             shadcn-generated primitives – do not hand-edit
    shared/         Components used across portals
    admin/, applicant/ Portal-specific components
  schemas/          Zod validation schemas, one module per domain
  hooks/            TanStack Query hooks
  lib/              Shared libraries: Supabase clients, Xendit client, mailer,
                    payment-redirect-check
  utils/auth/       Role resolution + Server Action authorization guards
  types/supabase.ts Auto-generated Supabase DB types – never hand-edit
  __tests__/        Vitest test suite
  supabase/
    migrations/     SQL migration history (see §5)
  docs/             Turnover documentation package + the end-user manual
```

Key Design Decisions / Patterns

- **Server Actions over REST for app logic.** Only 3 real HTTP route handlers exist (see §6); everything else the UI needs — creating an application, reviewing a document, managing admins — is a Server Action in `src/actions/`, called directly from Server/Client Components. Naming convention: `xAction` (e.g. `loginAction`).
- **Role is derived server-side from the database, not trusted from the JWT alone.** A user's role (`applicant / admin / super_admin`) is resolved by checking `super_admin_profiles` then `admin_profiles.is_active` in `src/utils/auth/getUser.ts` — the same logic is duplicated in `middleware.ts` and the auth callback route, since `user_metadata.role` on the Supabase session can be stale or spoofed client-side.
- **Three tiers of authorization enforcement:**
 1. **Middleware** (`src/lib/supabase/middleware.ts`) — redirects users away from route prefixes (`/applicant`, `/admin`, `/super-admin`) their role doesn't match.
 2. **Page/layout guards** — `assertRole()` redirects to `/unauthorized`.
 3. **Server Action guards** — `requireAdmin()` / `requireSuperAdmin()` / `requireApplicant()` / `withAdmin()` / `withSuperAdmin()` return a typed `{ authorized: false, error }` result instead of throwing, so actions can surface a proper error to the UI.
- **Zod validation at the Server Action boundary.** Schemas live in `src/schemas/`, validated with `.safeParse()` inside the action before touching the database.
- **revalidatePath before redirect** is a documented convention (`AGENTS.md`) — since `redirect()` throws, any cache revalidation must happen first or it's skipped.
- **Two Supabase clients:** a request-scoped client (`createClient`, publishable key + user cookies, subject to RLS) and an admin client (`createAdminClient`, service-role key, bypasses RLS) used only in trusted server contexts (webhooks, role lookups, admin actions).
- **Payment status is intended to be provider-authoritative.** `payments.status` is only ever written by the Xendit webhook handler in application code — the admin UI can only *read* payment state.

Caveat. This is an application-layer convention, not a database-enforced one — the live RLS policy on `payments` actually lets an authenticated user update their own payment row directly. See §5 and §12.

- **One active payment per user+service.** A partial unique index guarantees a user can't have two simultaneous `pending / success` payments for the same service, and the webhook handler has an explicit dedup guard against double-counting a paid invoice.

3. Deployment Guide

Hosting, build commands, deployment steps, rollback procedure, and domain/DNS configuration.

Hosting Provider

Vercel. The app is a standard Next.js project with no `vercel.json` overrides — Vercel's default Next.js build/runtime detection applies.

To verify. Confirm in the Vercel dashboard (Project → Settings) whether the branch → environment mapping described below is actually configured that way. This guide was written without live Vercel access at the time — the CLI was logged out, and login is an interactive browser step that has to be done by whoever holds the account.

Build Commands

```
pnpm install --frozen-lockfile
pnpm build      # runs `next build`
```

Local dev: `pnpm dev`. Production start (only relevant for non-Vercel runs): `pnpm start`.

The `pnpm build` step needs these environment variables available at build time (see §4 for the full list and purpose):
`NEXT_PUBLIC_SUPABASE_URL`, `NEXT_PUBLIC_SUPABASE_ANON_KEY`, `NEXT_PUBLIC_SUPABASE_PUBLISHABLE_KEY`,
`SUPABASE_SECRET_KEY`, `XENDIT_SECRET_KEY`, `XENDIT_WEBHOOK_TOKEN` — these must be set as Vercel Project → Environment Variables for each environment (Production / Preview / Development), the same way they're set as GitHub Actions secrets for CI.

Deployment Steps

Branch flow (confirmed by the team): every change is promoted through three branches via PR —

```
feature/* → PR → develop  (dev/QA)
develop   → PR → staging   (live testing)
staging    → PR → main     (production)
```

`.github/workflows/promote-staging.yml` provides a manual `workflow_dispatch` job that opens the `develop → staging` promotion PR automatically (`gh pr create`) — run it from the Actions tab rather than opening that PR by hand.

CI (`.github/workflows/ci.yml`) runs on every PR into `develop`, `staging`, or `main`: `pnpm install --frozen-lockfile → pnpm tsc --noEmit → pnpm test:coverage → pnpm build`. This is a gate, not a deploy — it does not push anything live.

Deploys to Vercel are not wired into GitHub Actions. `.github/workflows/deploy-production.yml` exists but is entirely commented out. In practice, deploys almost certainly happen through **Vercel's own native GitHub integration** — Vercel watches the connected repo directly and builds/deploys on push, independent of the Actions workflows above. Confirm this in Vercel Project → Settings → Git, and note there which branches map to which Vercel environment (typically `main` → Production, other branches → Preview deployments).

Manual fallback, if ever needed and the CLI is authenticated:

```
vercel login
vercel link      # first time only, links this repo to the Vercel project
vercel deploy --prod # deploy the current branch to Production
```

Rollback Procedure

Vercel keeps every deployment immutable and browsable. To roll back:

1. Vercel dashboard → Project → **Deployments**.
2. Find the last known-good deployment (by commit/branch).
3. Click `⋮` → **Promote to Production** (Vercel's "Instant Rollback") — this repoints the production domain to that build with no rebuild required.
4. Alternatively via CLI: `vercel rollback [deployment-url]`.

No database rollback tooling exists in this project — a Vercel rollback only affects application code, not Supabase schema/data. See §5 for schema history and §10 for backup/restore.

Domain / DNS Configuration

Setting	Value
Production domain	<code>retirewellsrrv.com</code>
Registrar	Porkbun

Setting	Value
Points to	Vercel — standard setup: an <code>A / ALIAS</code> record for the apex domain and/or a <code>CNAME</code> for <code>www</code> pointing at Vercel's edge, configured from Vercel Project → Settings → Domains, which supplies the exact record values to enter in Porkbun.

`next.config.mjs` also lists `srv.vercel.app` as a known dev origin, suggesting the Vercel project itself is internally named `srv` — that's the default Vercel subdomain, separate from the custom `retirewellsrv.com` domain.

To verify/reconfigure: Vercel dashboard → Domains (shows exact DNS records expected) cross-checked against Porkbun's DNS management panel for `retirewellsrv.com`.

4. Environment Variables

Names and purposes of every environment variable the application uses.

Actual secret values belong in the credentials vault — see §8 — never in this document. Template file in the repository: `env.copy.example`.

Variables

Variable	Purpose	Used in	Required for
<code>NEXT_PUBLIC_SUPABASE_URL</code>	Supabase project URL	<code>src/lib/supabase/{client,server,middleware}.ts</code>	Build + runtime
<code>NEXT_PUBLIC_SUPABASE_ANON_KEY</code>	Legacy Supabase anon/public key	<code>src/lib/supabase/middleware.ts</code>	Build + runtime (middleware only)
<code>NEXT_PUBLIC_SUPABASE_PUBLISHABLE_KEY</code>	Current Supabase "publishable" key for browser/server (non-admin) clients	<code>src/lib/supabase/client.ts</code> , <code>server.ts</code> (<code>createClient</code>)	Build + runtime
<code>SUPABASE_SERVICE_ROLE_KEY</code>	Privileged server-only key, bypasses Row Level Security	<code>src/lib/supabase/server.ts</code> (<code>createAdminClient</code>)	Runtime (server only — never expose to the browser)
<code>SUPABASE_SECRET_KEY</code>	Present in env files / CI secrets, but no code reference found in <code>src/</code> — appears unused currently, possibly reserved for a future Supabase key-naming migration	—	Set in CI/Vercel for parity, but not currently consumed
<code>NEXT_PUBLIC_SITE_URL</code>	Base app URL — used to build OAuth/email-confirmation redirect links and for payment-redirect health checks	<code>src/actions/auth.ts</code> , <code>admins.ts</code> , <code>payment-redirect-check.ts</code>	Runtime
<code>RESEND_API_KEY</code>	Resend transactional email API key	<code>src/lib/mailer.ts</code>	Runtime — without it, emails are skipped and logged as <code>skipped in email_logs</code>
<code>ADMIN_EMAIL</code>	Destination/from address for admin-facing notification emails (defaults to <code>dev@retirewellsrrv.com</code> if unset)	<code>src/lib/mailer.ts</code>	Runtime
<code>XENDIT_SECRET_KEY</code>	Xendit payment gateway secret key — creates invoices	<code>src/lib/xendit.ts</code>	Runtime
<code>XENDIT_WEBHOOK_TOKEN</code>	Shared secret validating incoming Xendit webhook calls	<code>src/app/api/webhooks/xendit/route.ts</code>	Runtime — payment status updates are silently rejected (401) without it
<code>VERCEL_TOKEN</code>	Vercel deploy token	CI only (currently unused — deploy job is disabled)	CI/CD, if the deploy workflow is re-enabled
<code>VERCEL_ORG_ID</code>	Vercel organization ID	CI only (currently unused)	CI/CD, if re-enabled

Variable	Purpose	Used in	Required for
VERCEL_PROJECT_ID	Vercel project ID	CI only (currently unused)	CI/CD, if re-enabled

Production vs. Development — Don't Mix These Up

`.env.production` points at `kpmzzxbbethfqqrqacd.supabase.co` — the real Supabase project linked to the client's account, and the one all the other turnover documents (schema, RLS, etc.) are based on.

`.env.local` and `.env.staging` both point at a **different** project, `hopxdottqixfacifklka.supabase.co` — the personal/development project used during development. Values from those two files (or anything read from that project's dashboard) are **not** representative of the client's production data or schema. When setting Vercel environment variables or troubleshooting production, always work from `.env.production` / the Vercel Production environment, not the Preview/Development ones.

Where These Need to Be Set

- **Local development:** `.env.local` (copy from `env.copy.example`).
- **CI (GitHub Actions):** repository Settings → Secrets and variables → Actions. `ci.yml` currently consumes: `NEXT_PUBLIC_SUPABASE_URL`, `NEXT_PUBLIC_SUPABASE_ANON_KEY`, `NEXT_PUBLIC_SUPABASE_PUBLISHABLE_KEY`, `SUPABASE_SECRET_KEY`, `XENDIT_SECRET_KEY`, `XENDIT_WEBHOOK_TOKEN`.
- **Vercel (build + runtime):** Project → Settings → Environment Variables, set per environment (Production / Preview / Development). See §3.

Security Note

`.env.local`, `.env.production`, and `.env.staging` exist at the repository root with live values and are correctly listed in `.gitignore` — verified not tracked in git history as of this handover. Keep it that way: never force-add an env file to git.

5. Database Schema / ERD

Tables, relationships, enums, RLS policies, and migration history for the SRRV production database.

Source of truth: a real supabase db dump of the production project (found at docs/backup2/ — schema.sql, data.sql, roles.sql). Cross-checked against src/types/supabase.ts — the two agree on all 20 tables and their relationships.

Confirmed production project: ref kpmzzxzbbethfqqrqacd — this matches the NEXT_PUBLIC_SUPABASE_URL in .env.production. .env.local and .env.staging both point to a different project (ref hopxdottqixfacifklka) — the personal/dev project used during development, not what's documented here.

Tables

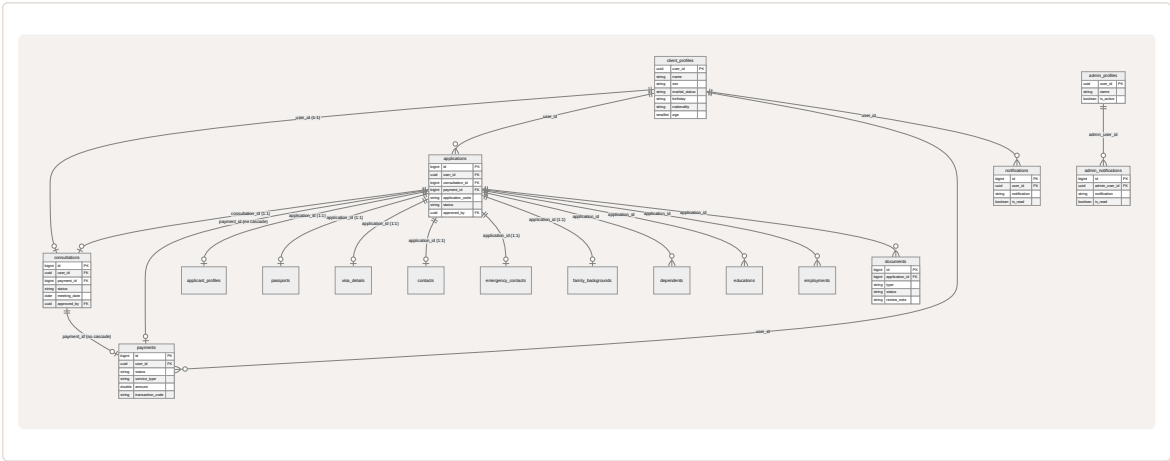
20 tables in the public schema, all keyed off Supabase's built-in auth.users.

Table	Purpose
client_profiles	One row per applicant: name, sex, birthday, nationality, marital status (default single). Primary applicant identity record.
admin_profiles	One row per admin: name, is_active (default true) — disables login/notifications without deleting the account.
super_admin_profiles	One row per super admin: name. Presence in this table overrides any other role.
consultations	An applicant's consultation request: meeting date, communication mode, purpose, status (default pending), linked payment, approved_by / approved_at.
applications	The SRRV application itself: application code, status (default pending), linked consultation and payment, approved_by / approved_at.
applicant_profiles	Detailed personal info for one application (civil status, DOB, nationality, physical description).
passports	Passport details for one application.
visa_details	Entry visa / tourist visa details for one application.
contacts	Contact info (addresses, phone, email) for one application.
emergency_contacts	Emergency contact for one application.
dependents	Dependents/family members included in an application (many per application).
family_backgrounds	Parents' info for one application.
educations	Education history entries for an application (many per application).
employments	Employment history entries for an application (many per application).
documents	Uploaded files for an application (many per application) — status (default pending), type, format, reviewer note.
payments	A single payment transaction (consultation or application fee) — amount, status, method, transaction code.
notifications	In-portal notifications for an applicant.
admin_notifications	In-portal notifications for an admin.
email_logs	Audit log of every outbound email attempt (sent/failed/skipped).
service_plans	Public-facing service/package listings shown on the marketing site.

All primary keys are bigint identity columns except the four profile tables, which use the Supabase Auth uuid as their primary key directly.

Relationships

Every foreign key below is ON DELETE CASCADE unless noted otherwise.



Single-row-per-application tables and multi-row child tables are simplified in the boxes above for readability, but their FK relationships to `applications` are shown. `service_plans` and `email_logs` have no foreign keys. `applications.approved_by` and `consultations.approved_by` also reference `auth.users` directly — not shown, since `auth.users` lives outside the `public` schema.

Enums

Enum	Values
<code>application_status</code>	<code>processing</code> , <code>paused</code> , <code>approved</code> , <code>rejected</code> , <code>pending</code> , <code>payment_failed</code>
<code>consultation_status</code>	<code>processing</code> , <code>accepted</code> , <code>rejected</code> , <code>pending</code>
<code>document_status</code>	<code>processing</code> , <code>accepted</code> , <code>rejected</code> , <code>action need</code> , <code>pending</code>
<code>document_type</code>	<code>passport</code> , <code>photo_2x2</code> , <code>pra_application</code> , <code>police</code> , <code>medical</code> , <code>bicc</code> , <code>bank_cert</code> , <code>proof_payment</code> , <code>proof_pension</code> , <code>proof_relationship</code>
<code>document_format</code>	<code>pdf</code> , <code>doc</code> , <code>docx</code> , <code>jpg</code> , <code>png</code> , <code>gif</code> , <code>bmp</code> , <code>webp</code> , <code>tiff</code> , <code>tif</code> , <code>jpeg</code>
<code>payment_status</code>	<code>processing</code> , <code>pending</code> , <code>cancelled</code> , <code>success</code> , <code>failed</code>
<code>payment_methods</code>	<code>pool</code> , <code>callback_virtual_account</code> , <code>credit_card</code> , <code>retail_outlet</code> , <code>qr_code</code> , <code>qris</code> , <code>ewallet</code> , <code>direct_debit</code> , <code>bank_transfer</code> , <code>paylater</code> , <code>cryptocurrency</code>
<code>service_type</code>	<code>application</code> , <code>consultation</code>
<code>communication_mode</code>	<code>zoom_meeting</code> , <code>google_meet</code> , <code>whatsapp</code> , <code>face_2_face</code> , <code>phone_call</code>
<code>marital_status</code>	<code>single</code> , <code>married</code> , <code>widowed</code> , <code>divorced</code>
<code>sex</code>	<code>male</code> , <code>female</code>

Database Functions

Function	Purpose
<code>public.insert_client_profile()</code>	Trigger function: on a new <code>auth.users</code> row, creates an <code>admin_profiles</code> row if <code>raw_user_meta_data->>'role' = 'admin'</code> , otherwise creates a <code>client_profiles</code> row seeded from signup metadata. The dump only captures <code>public</code> -schema objects, so the actual <code>CREATE TRIGGER ... ON auth.users</code> attaching this function isn't visible — worth confirming directly in the Supabase SQL editor that it's still attached.
<code>public.set_updated_at()</code>	Standard <code>NEW.updated_at = NOW()</code> trigger function. Same caveat about trigger attachment visibility.

Row Level Security — Pattern and Exceptions

Most tables (`applications`, `consultations`, `contacts`, `dependents`, `documents`, `educations`, `emergency_contacts`, `employments`, `family_backgrounds`, `passports`, `visa_details`, `client_profiles`) follow the same RLS pattern: **owner (user_id / via their application) OR any row in admin_profiles OR any row in super_admin_profiles** can select/insert/update/delete. `notifications` and `admin_notifications` are scoped strictly to their own `user_id` / `admin_user_id`. `admin_profiles`, `super_admin_profiles`, and `email_logs` have RLS enabled with no or near-no policies — access goes through the service-role admin client only.

Two things worth flagging (also listed in §12):

1. **payments** RLS allows an authenticated user to INSERT/UPDATE/SELECT their own payment row directly — no restriction on which columns or values. The application's own Server Actions/webhook never let a user set their own payment to `success`, but RLS doesn't stop a client from calling the Supabase REST/JS API directly to do exactly that.
2. The **"is admin" RLS check doesn't look at `admin_profiles.is_active`**. The app's own authorization treats a disabled admin as a plain applicant, but the RLS policies only check row *existence* in `admin_profiles`, not `is_active`. A disabled admin who retains a valid session could still read/write anything the "any admin" RLS branch allows.

Neither of these was caught by application-level testing since the app never issues the requests that would exploit them — they only show up by reading the RLS policy definitions directly.

Indexes

Index	Table	Notes
<code>payments_one_active_per_user_service_idx</code>	<code>payments</code>	Unique, partial (<code>WHERE status IN ('pending', 'success')</code>) — one active payment per user+service.
<code>admin_notifications_admin_user_id_idx</code>	<code>admin_notifications</code>	(<code>admin_user_id, id DESC</code>) — feed lookups.
<code>email_logs_created_at_idx</code>	<code>email_logs</code>	(<code>created_at DESC</code>) — audit log browsing.

Realtime

`supabase_realtime` publication includes `admin_notifications` and `notifications` — powers the live notification bell in both portals.

Migration History / Notes

Location: `supabase/migrations/`

Migration	Date	What it does
<code>20260526144951_initial_migration.sql</code>	2026-05-26	Defines an early/legacy schema — <code>SERIAL</code> integer user IDs, a hand-rolled <code>users</code> table, different enum names/values, a <code>messages</code> table that no longer exists. Does not match the live schema.
<code>20260811000002_notifications_rls_realtime.sql</code>	2026-08-11	Adds <code>notifications</code> / <code>admin_notifications</code> to Realtime; enables RLS with user-scoped policies. Matches the live dump.
<code>20260813000001_email_logs.sql</code>	2026-08-13	Creates <code>email_logs</code> . Matches the live dump.
<code>20260814000001_unique_active_payment.sql</code>	2026-08-14	Adds <code>payments_one_active_per_user_service_idx</code> . Matches the live dump.

Gap: only the first migration is out of date — everything from 2026-08-11 onward matches the live dump exactly. The bulk of the schema (all the application/profile/document tables and their RLS policies) predates the earliest migration file and was evidently built directly via the Supabase dashboard/SQL editor before migration tracking started. Recommend committing a trimmed version of the production `schema.sql` as a baseline migration so `supabase migration list` and the real schema agree going forward.

Storage

Supabase Storage bucket `documents` holds applicant-uploaded files, referenced by `documents.path`. Upload/access logic: `src/actions/applicant/document.ts`, `src/actions/applicant/application.ts`.

6. API Documentation

The application's HTTP endpoints and Server Actions.

This app is a Next.js App Router application. Its real "API" surface for the UI is almost entirely **Server Actions** (`src/actions/`), not REST endpoints — these are called directly from React components over Next.js's internal RPC mechanism, not `fetch` , and have no URL, method, or public contract in the traditional sense. Only **3 genuine HTTP route handlers** exist under `src/app/api/` . Both are documented below; a companion OpenAPI file (`06-api-collection.json`) covers just those 3 routes, since that's what OpenAPI/Postman are meant to describe.

REST Endpoints

GET /API/AUTH/CALLBACK

File: `src/app/api/auth/callback/route.ts`

Purpose: Supabase Auth callback — completes an OAuth login (`?code=`) or an email-confirmation link (`?token_hash=&type=email`). Assigns the default `applicant` role and seeds a `client_profiles` row on first login, checks `super_admin_profiles` to override role, and redirects to the correct role dashboard (or to profile completion if required fields are missing).
Auth: None required to call it — it *establishes* the session from the Supabase code/token itself. Only relative `next` redirect targets are honored (prevents open-redirect abuse).
Query params: `code` (OAuth), `token_hash + type=email` (email confirmation), `next` (optional relative redirect override).
Response: HTTP redirect (302), never a JSON body.

POST /API/PAYMENTS/INVOICE

File: `src/app/api/payments/invoice/route.ts`

Purpose: Creates a Xendit hosted-checkout invoice for the logged-in user.
Auth: Required — validates `supabase.auth.getUser()` ; returns 401 if not authenticated.
Request body (validated with `createInvoiceSchema`):

```
{
  "amount": 300,
  "description": "SRRV application fee",
  "payerEmail": "optional, defaults to the logged-in user's email",
  "successRedirectUrl": "optional",
  "failureRedirectUrl": "optional",
  "paymentMethods": ["optional array of strings"],
  "currency": "optional, 3-letter code",
  "metadata": { "optional": "string map" }
}
```

Response 200:

```
{ "success": true, "data": { "id": "...", "invoiceUrl": "...", "externalId": "...", "amount": 300, "status": "PEND
```

401: { "success": false, "error": "Unauthorized" } **400:** invalid JSON or failed validation **500:** Xendit invoice creation failed.

POST /API/WEBHOOKS/XENDIT

File: `src/app/api/webhooks/xendit/route.ts`

Purpose: Receives payment status callbacks from Xendit and updates the matching `payments` row. Also sends a confirmation email to the admin mailbox when a consultation payment succeeds.
Auth: Shared-secret token, not a session — the `x-callback-token` (or `Xendit-Webhook-Token`) header must match `XENDIT_WEBHOOK_TOKEN` ; returns 401 otherwise. This is the endpoint to register as the webhook URL in the Xendit dashboard.
Safeguards: only updates payments still in `pending` status (a late `EXPIRED / FAILED` event can never downgrade an already- `success` payment); dedups so a duplicate `PAID` event for the same user+service can't create two successful payments.
Response: always { `"received": true` } on 200, or 401/400 for auth/parsing failures.

Server Actions (not REST — listed for completeness)

Module	File	Covers
Auth	<code>src/actions/auth.ts</code>	Login, register, password reset
Admin — dashboard	<code>src/actions/admin/dashboard.ts</code>	KPI stats for the admin dashboard
Admin — consultations	<code>src/actions/admin/consultations.ts</code>	Accept/reject/process consultations
Admin — applications	<code>src/actions/admin/applications-admin.ts</code>	Review + approve/reject applications

Module	File	Covers
Admin — documents	documents.ts , document-review.ts	Document queue + review decisions
Admin — payments	src/actions/admin/payments.ts	Read-only payment monitoring
Admin — client profiles	client-profiles.ts	Client register, profile correction
Admin — services	src/actions/admin/service.ts	Service/package CRUD
Admin — notifications	notifications.ts	Mark read / clear admin notifications
Admin — admin/user mgmt	admins.ts , users.ts	Create/disable/delete admin accounts (super admin only)
Applicant — application	src/actions/applicant/application.ts	The multi-step SRRV application form
Applicant — consultation	consultation.ts	Request/update a consultation
Applicant — document	document.ts	Upload/re-upload documents
Applicant — profile	profile.ts	Personal details
Applicant — notifications	notifications.ts	Mark read / clear notifications

Every admin-scoped action is guarded by `requireAdmin()` / `requireSuperAdmin()` / `withAdmin()` / `withSuperAdmin()` from `src/utils/auth/getUser.ts` and `with-admin.ts`; applicant actions by `requireApplicant()`.

Full request/response schema also available as OpenAPI 3.0: `docs/06-api-collection.json`.

7. Access Handover Sheet

Every third-party account the platform depends on, and who owns it going forward.

Per the client's confirmation: **all accounts below are owned by the client going forward.**

Service	Purpose	Owner	Account status
Vercel	Application hosting/deployment	Client	Placeholder — confirm login works
Porkbun	Domain registrar / DNS for retirewellsrvv.com	Client	Placeholder — confirm login works
Supabase	Database, Auth, Storage — production project ref kpmzzxzbbethfqqrqacd	Client	Placeholder — confirm login works. Do not confuse with the dev project hopxdottqixfacifklka used in .env.local / .env.staging , which is not the client's.
Xendit	Payment gateway	Client	Placeholder — confirm login works
Resend	Transactional email	Client	Placeholder — confirm login works
Analytics/monitoring	—	N/A	None configured. No Sentry, Google Analytics, Vercel Analytics, or similar tooling exists in the codebase. Confirmed expected — not a gap.
GitHub repository	Source code	Client	Done — repository already transferred to the client's GitHub account; they log in with their own email.

Notes

- Per the contract (§7 Client Responsibilities), the client is responsible for all third-party subscription costs (Vercel, Supabase, Xendit transaction fees, domain registration) going forward.
- Actual credential values are not in this document — see §8 for how those move to the client.
- Placeholder rows above should be checked off once someone has actually logged into each service with the client's own credentials.

8. Credentials Transfer Checklist

A procedure to follow, not a place secrets are stored.

No live credential values are handled in this document or by any automated process — everything below is a manual step for whoever currently holds each account.

1. Set Up a Shared Vault

- Create a shared vault (e.g. **Bitwarden** — an Organization or a shared Collection so access can be revoked per-person later) that both the provider team and the client can access.
- Invite the client's designated account holder.

2. Entries to Create in the Vault

- **Supabase** — project dashboard login (or org invite), plus the 4 keys/URLs currently in `.env.production`:
`NEXT_PUBLIC_SUPABASE_URL`, `NEXT_PUBLIC_SUPABASE_PUBLISHABLE_KEY`, `SUPABASE_SERVICE_ROLE_KEY`,
`SUPABASE_SECRET_KEY`
- **Vercel** — dashboard login (or team invite), plus `VERCEL_TOKEN` / `VERCEL_ORG_ID` / `VERCEL_PROJECT_ID` if CI deploys are ever re-enabled
- **Xendit** — dashboard login, `XENDIT_SECRET_KEY`, `XENDIT_WEBHOOK_TOKEN`
- **Resend** — dashboard login, `RESEND_API_KEY`
- **Porkbun** — registrar account login for `retirewellsrrv.com`
- **GitHub (done)** — repository already transferred to the client's GitHub account (they log in with their own email); no vault entry needed beyond noting this is done
- `ADMIN_EMAIL` mailbox credentials (whatever inbox receives staff notification emails)

3. Move the Values

- Copy each value out of `.env.production` (and the equivalent Vercel dashboard entries) into its vault entry.
- **Rotate every key/secret after transfer** — anyone who had prior access (contractors, former team members) should not retain working credentials. This is standard practice for a handover, not a sign anything leaked.
- Delete or restrict any old vault/password-manager entries that held these values pre-transfer.

4. Confirm the Client Can Actually Log In

- Client can access Supabase project dashboard
- Client can access Vercel project dashboard
- Client can access Xendit dashboard
- Client can access Resend dashboard
- Client can access Porkbun account
- Client can access the GitHub repository with appropriate permissions (**already transferred**)

Only mark item 8 complete once every item above is checked off.

10. Maintenance Guide

Common issues, how to check logs, how to restart services, and backup/restore.

Common Issues and Fixes

See §12 for the full list of known limitations and cleanup items. Highlights relevant to day-to-day maintenance:

- **A fee amount needs to change** — ₱50/₱300 fees are hardcoded in source, not admin-configurable. Requires a code change and redeploy (see §3).
- **A payment looks stuck as "pending"** — payment status is only ever updated by the Xendit webhook (`src/app/api/webhooks/xendit/route.ts`). If a payment never transitions, check that Xendit's webhook is actually configured to call `/api/webhooks/xendit` with the correct `XENDIT_WEBHOOK_TOKEN` , and check Xendit's own dashboard for the invoice's real status.
- **A refund was issued** — there's no in-portal refund action; refunds happen directly in the Xendit console. The portal will only reflect it once the payment is cancelled there.
- **Emails aren't sending** — check `RESEND_API_KEY` is set; if missing, sends are skipped (not retried) and logged as `skipped` in the `email_logs` table.

How to Check Logs

Source	Where
Application/server logs (Server Actions, API routes)	Vercel dashboard → Project → Logs (or Deployments → [deployment] → Functions) for real-time and recent runtime logs
Build logs	Vercel dashboard → Project → Deployments → [deployment] → Build Logs
Database/Auth logs	Supabase dashboard → Project → Logs (separate sections for Postgres, Auth, API)
Email delivery attempts	<code>email_logs</code> table in Supabase — every send attempt (sent/failed/skipped) with the error message if any. Query via Supabase Table Editor or SQL Editor.
Payment events	Xendit dashboard → Invoices — shows the authoritative status of every invoice, independent of what's recorded locally

How to "Restart" Services

There are no long-running servers to restart — this is a serverless Next.js app on Vercel.

- **App:** trigger a new deployment (push a commit, or Deployments → ⋯ → Redeploy in the Vercel dashboard) — this is the equivalent of a restart.
- **Database:** Supabase is a managed Postgres instance; nothing to restart under normal operation. If the project appears paused/inactive (e.g. on a free tier after inactivity), resume it from the Supabase dashboard.

Backup and Restore

Supabase provides automated backups on paid plans (daily backups + optional Point-in-Time Recovery, depending on plan tier). **Confirm the actual plan and retention window in the Supabase dashboard → Project Settings → Backups** for the production project (ref `kpmzzxzbbethfqrrqacd` , matching `.env.production`) — this wasn't verified live during this documentation pass since it requires an interactive dashboard login. A schema-and-data dump of this project already exists at `docs/backup2/` as of the dump date, which can serve as a point-in-time reference independent of Supabase's own backup retention.

To restore: Supabase dashboard → Database → Backups → select a backup point → restore. This is a destructive operation on the target project — confirm with the client before triggering one.

For file storage (`documents` bucket), Supabase Storage backups follow the same project-level backup policy as the database.

11. Source Code Repository Access

Ownership and setup instructions for the SRRV codebase.

Status: Done. Repository already transferred to the client's GitHub account. The client logs in with their own email/account — no separate credential to hand over for this one.

Remaining Housekeeping

- Confirm branch protection rules on `main / staging / develop` still make sense under the new access model (who can merge, required checks, etc.).
- Revoke access for any contractors/former team members no longer involved, once the client confirms they're fully set up (unless retained for the post-launch support window — see §13).

README

- Already covers setup instructions — clone, `pnpm install`, env var setup, dev server, DB type generation, migrations. See `README.md` at the repository root. No changes needed for turnover.

12. Known Issues / Limitations

Outstanding items, code-level cleanup, and intentionally deferred scope.

Security Findings (from Reading the Live Database's RLS Policies Directly)

These were found by reading the actual RLS policy SQL in the production database dump (docs/backup2/schema.sql), not by testing the app's UI — the app's own code never exercises these paths, which is exactly why they're easy to miss.

Item	Detail	Location
A user can set their own payment to "success" without paying	The payments table's RLS policies let an authenticated user INSERT/UPDATE/SELECT their own payment row with no restriction on which columns or values — including status . The app's Server Actions and webhook handler never expose this, but nothing in the database stops a user from calling the Supabase client directly to mark their own payment as success , bypassing Xendit entirely. Recommended fix: restrict the UPDATE policy to only the columns a user should legitimately touch (or drop it entirely and let all writes go through the service-role webhook).	docs/backup2/schema.sql payments policies; see §5
A disabled admin's RLS access doesn't actually get revoked	The "is this user an admin" RLS check used across most tables never checks admin_profiles.is_active — only row existence. The app's own authorization correctly treats a disabled admin as an applicant, but that's an application-layer check; the database-layer RLS would still grant admin-level access to a disabled admin who retains a valid session. Recommended fix: change the RLS admin-check subqueries to also filter is_active = true .	docs/backup2/schema.sql — every table policy
Password logged in plaintext	The password-reset flow logs the new password to the server console via console.log . Should be removed before production traffic.	src/actions/auth.ts:284

Outstanding Items to Resolve Before/Soon After Go-Live

Item	Detail	Location
Leftover debug logging	Several console.log calls print user IDs, roles, and full user/session objects during auth and password-reset flows. Noisy in production logs; one of them dumps a full user object.	src/lib/supabase/middleware.ts:51-53 , src/actions/auth.ts:109-111,232-234,247
"Fill Test Data" dev helper	A button that autofills the application form with test data is still visible on steps 1–2 of the live application form. Should be removed before real applicants use the portal.	src/app/(protected)/applicant/application/page.tsx:222
Fee amounts are hardcoded	The ₱50 consultation fee and ₱300 application fee are set in application source code, not an admin-configurable setting. Changing a fee requires a code change + redeploy.	Referenced in the User Manual §3.2
No in-portal refund control	Refunds must be issued directly in the Xendit console. The admin portal only reflects cancelled payments as "refunded" in its statistics.	Referenced in the User Manual Appendix D
Payment method is provisional at creation	payments.payment_method is set to a placeholder value when the row is created and only corrected to the real method once Xendit's webhook callback arrives. Expected behavior, not a bug.	src/app/api/webhooks/xendit/route.ts

Code-Level Cleanup Items (Not User-Facing, but Worth Doing)

Item	Detail
SUPABASE_SECRET_KEY env var is unused	Present in .env* files, env.copy.example , and CI secrets, but no code reference exists anywhere in src/ . Either reserved for a future Supabase key-naming migration, or dead config.
src/lib/supabase/database.types.ts is an empty leftover file	0 lines. Superseded by src/types/supabase.ts , which is the one actually imported throughout the app. Safe to delete once confirmed nothing references it.

Item	Detail
First migration file is stale	<code>supabase/migrations/20260526144951_initial_migration.sql</code> describes an early schema design that doesn't match the live database. See §5 for the reconciliation recommendation.
CI deploy job is fully commented out	<code>.github/workflows/deploy-production.yml</code> — production deploys currently happen outside of GitHub Actions (presumably Vercel's native Git integration). See §3.

Intentionally Deferred (Out of Contracted Scope)

Per the contract (§1 Scope of Work), anything not listed as a delivered module is out of scope and would require a separate written agreement — e.g. no analytics/monitoring module was built (that's listed in the contract §8 as a paid add-on, PHP 15,000), and messaging is described as "chatbot-ready architecture," not a working chatbot.

13. Support / Warranty Terms

Sourced directly from the signed Project-Base Web Development Contract, §5.

Contract: Project-Base Web Development Contract — SRRV Service Platform, effective May 22, 2026 – July 30, 2026.

Free Support Window

45 calendar days of free post-launch support, starting from the date of system turnover (i.e. from the date this documentation package and full system access are handed over — not from the contract signing date).

What's Covered

Bug fixes **directly related to the delivered scope** — the modules listed in contract §1 (Scope of Work): public site, auth/RBAC, applicant dashboard, application form, document upload, application tracking, payment module, messaging/chatbot-ready architecture, admin dashboard, service/package management, notifications, infrastructure/deployment.

What's Not Covered

- New features or functionality not in the original scope (contract §1 — these require a separate written agreement and quotation).
- Issues arising from third-party service outages (Vercel, Supabase, Xendit, etc.) — contract §10.
- Data loss due to the client's failure to maintain proper backups after turnover — contract §10.
- Security issues caused by the client's own misuse of credentials or failure to follow security recommendations — contract §10.
- Revisions beyond the 2 free revisions already used pre-launch (contract §4 — additional revisions are PHP 3,000.00 each; separate from the post-launch support window).

After the Free Support Window Ends

Two optional paths, per contract §5 (must be confirmed in writing to activate either):

Option	Cost	Covers
Monthly Retainer	PHP 3,000.00/month	Routine monitoring, minor fixes, dependency updates. New features/major changes quoted separately.
Ad-Hoc / Pay-Per-Fix	PHP 1,500.00 per reported issue	For clients who don't want an ongoing monthly commitment.

Neither is automatic — the contract requires written confirmation to start either arrangement.

Contact Process for Support Requests

The contract's convention for change/revision requests (§4) is a written request via email or the agreed messaging platform, clearly describing the issue. The same process applies to support requests during and after the free window.

Related Terms

- **Liability is capped** at the total amount paid under the contract (§10).
- **Confidentiality** obligations continue to apply to both parties regarding credentials and sensitive data shared during the project (§9).

Companion Documents

These are separate, standalone documents — not part of this file:

#	Document	File
1	User Guide	SRRV-User-Manual.html / .pdf
9	Admin & Super Admin Guide	09-admin-user-guide.html / .pdf
14	Sign-off / Acceptance Certificate	14-sign-off-acceptance-form.html / .pdf — the official document to print and sign

This document, the API collection (06-api-collection.json), and the three companion documents above together make up the full turnover package.